Employee Attrition Prediction System

Project Report

By

Samruddhi Dube- AF05102782

Janvi Mhamunkar- AF05102827

Index

Sr. No	Topic	Page No
1.	Title of Project	1
2.	Acknowledgement	3
3.	Abstract	4
4.	Introduction	5-8
5.	System Analysis	9-21
6.	System Design	22-27
7.	Screenshots	28-32
8.	Implementation	33-36
9.	Results and Discussion	37-40
10.	Conclusion	41-42
11.	Future Scope	43
12.	Bibliography and References	44

Acknowledgment

The project **"Employee Attrition Prediction System"** is the Project work carried out by

Name	**Enrollment-No**
Samruddhi Dube	AF05102782
Janvi Mhamunkar	AF05102827

We are thankful to our project guide for guiding us to complete the Project.

Their suggestions and valuable information regarding the formation of the Project Report have provided us a lot of help in completing the Project and its related topics.

Abstract

Employee attrition is a major concern for organizations as it impacts productivity and increases operational costs. This project presents an Employee Attrition Prediction System that uses machine learning to identify employees who are likely to leave the organization based on factors such as job satisfaction, salary, and work experience.

The system is developed using React for the frontend, Spring Boot for backend services, Python for the prediction model, and MySQL for data storage. It provides a user-friendly interface for managing employee data and generates real-time predictions to support decision-making.

This solution helps organizations take proactive steps to improve employee retention by leveraging data-driven insights.

1.Introduction

Managing employee retention efficiently is a critical aspect of organizational success. In today's competitive business environment, organizations face significant challenges in identifying employees who are likely to leave. Employee attrition leads to increased recruitment costs, loss of skilled talent, and decreased productivity. Traditional methods such as manual analysis and basic reporting are time-consuming, error-prone, and lack predictive capabilities required for modern workforce management.

To overcome these limitations, technology-driven solutions using machine learning can play a vital role in predicting and reducing employee attrition. The Employee Attrition Prediction System is a web-based application developed using React, Spring Boot, Python, and MySQL, designed to analyze employee data and predict the likelihood of attrition.

The system integrates a machine learning model built using Python and Scikit-learn, where a Random Forest Classifier is used to provide accurate predictions. The application includes modules for employee data management, prediction processing, and result visualization, ensuring smooth data flow across frontend, backend, and machine learning components.

1.1 Background of the Study

Employee attrition is a major concern across various industries such as IT, healthcare, finance, and education. Organizations need effective strategies to retain skilled employees and maintain workforce stability. Traditional HR systems rely heavily on manual data analysis, spreadsheets, and reports, which often lead to inaccurate insights, delayed decisions, and inefficient resource management.

With advancements in data analytics and machine learning, organizations can now leverage predictive models to identify patterns

and trends in employee behavior. These models help in understanding the factors that influence attrition, such as job satisfaction, salary, overtime, and work-life balance. The Employee Attrition Prediction System is designed as a centralized digital platform that stores, processes, and analyzes employee data to generate meaningful predictions.

2. Motivation

The motivation behind developing this project arises from the growing need for an intelligent and automated system that can predict employee attrition effectively. The challenges observed in traditional employee management systems include:

• Lack of predictive analysis for identifying employees at risk

• Manual errors and inefficiency in handling large volumes of employee data

• Difficulty in analyzing multiple factors affecting employee retention

• Absence of a centralized system for data storage and real-time insights

This project addresses these challenges by integrating machine learning with modern web technologies to build a scalable and efficient system. It automates prediction processes and provides accurate insights for better decision-making.

3. Significance of the Study

This project holds significance for multiple stakeholders:

For Organizations: Helps in reducing attrition by identifying high-risk employees early

For HR Teams: Provides data-driven insights for better workforce management

For Employees: Enables organizations to improve work conditions and retention strategies

For Developers and Students: Demonstrates real-world integration of React, Spring Boot, Python, and MySQL

By introducing automation and predictive analytics, the system reduces manual workload, improves accuracy, and enhances organizational efficiency.

4. Features of Employee Attrition Prediction System

The system offers multiple modules that make employee management and prediction efficient and structured.

Employee Management

a) Add, update, view, and delete employee details such as name, salary, job role, and experience

b) Maintain a centralized database of employee records

c) Manage employee-related attributes required for prediction

Prediction Module

a) Analyze employee data using a machine learning model

b) Predict whether an employee is likely to leave or stay

c) Provide probability score and risk level (High, Medium, Low)

Frontend (React)

a) User-friendly interface for HR operations

b) Dynamic forms for employee data input

c) Display prediction results in real-time

Backend (Spring Boot)

a) Handle API requests and business logic

b) Communicate with Flask API for predictions

c) Ensure secure and efficient data processing

Machine Learning Module (Python + Flask)

a) Data preprocessing using Pandas and NumPy

b) Model training using Random Forest Classifier

c) Deployment using Flask API for real-time predictions

5. Reporting and Analytics

• Generate prediction results for employees

• Provide probability-based risk analysis (High, Medium, Low)

• Help HR teams take proactive decisions for employee retention

• Analyze key factors affecting attrition using model insights

2. System Analysis

System analysis is a crucial phase in the Software Development Life Cycle (SDLC). It involves identifying existing challenges in employee management, analyzing functional requirements, and evaluating the feasibility of the proposed solution. For the Employee Attrition Prediction System, this phase focuses on understanding HR requirements, employee data handling, and the need for predictive analytics to reduce attrition rates through a centralized and automated platform.

1. Problem Definition

Managing employee retention is a significant challenge faced by organizations in today's competitive environment. Traditional methods such as manual analysis, spreadsheets, and basic HR tools are inefficient and lack predictive capabilities. These approaches often lead to poor decision-making and increased employee turnover.

Some major problems include:

Data Inconsistency and Duplication: Employee data is often stored in multiple systems, leading to mismatches and redundancy.

Lack of Predictive Analysis: Traditional systems cannot identify employees who are at risk of leaving.

Inefficient Data Handling: Manual processes are time-consuming and prone to human errors.

Lack of Centralized System: Employee information and analysis are scattered, making management difficult.

2. Objectives of the System

The Employee Attrition Prediction System aims to:

• Provide a centralized platform to manage employee data and predictions

• Predict employee attrition using machine learning techniques

• Enable HR teams to identify high-risk employees early

• Reduce manual effort in analyzing employee data

• Ensure accurate and real-time prediction results

• Improve employee retention through data-driven insights

• Provide a scalable system for future enhancements such as analytics dashboards

3. Feasibility Study

Feasibility analysis evaluates the practicality of developing the proposed system.

a. Technical Feasibility

The system is developed using React for frontend, Spring Boot for backend, Python (Flask) for machine learning, and MySQL as the database. Machine learning is implemented using Scikit-learn with a Random Forest Classifier.

All technologies used are open-source and widely supported.

Conclusion: The project is technically feasible.

b. Economic Feasibility

The system uses open-source technologies such as React, Spring Boot, Python, and MySQL, resulting in minimal development cost.

It reduces employee turnover costs and improves efficiency.

Conclusion: The project is economically feasible.

c. Operational Feasibility

The system provides a user-friendly interface for HR teams to manage employee data and predictions.

It automates repetitive tasks and reduces human errors.

Conclusion: The system is operationally feasible.

d. Time Feasibility

The project can be developed within a defined timeframe using a modular approach (Frontend, Backend, ML model).

Each module can be developed and tested independently.

Conclusion: The project is time-feasible.

4. System Requirements

a. Functional Requirements

- Add, view, and delete employee data
- Store employee attributes such as salary, job role, and experience
- Predict employee attrition using machine learning model
- Display prediction results with probability and risk level
- Provide API integration between frontend, backend, and ML model
- Generate prediction-based insights for HR

b. Non-Functional Requirements

- Usability: Easy-to-use web interface

• Reliability: Accurate prediction using trained model

• Scalability: Can handle large employee datasets

• Security: Secure data handling and API communication

• Performance: Fast prediction and data retrieval

c. Hardware Requirements

• Processor: Intel i3 or higher

• RAM: Minimum 4 GB (8 GB recommended)

• Storage: 500 MB or more

d. Software Requirements

• Operating System: Windows

• Frontend: React

• Backend: Spring Boot (Java)

• Machine Learning Model: Random Forest

• Libraries: Pandas, NumPy, Scikit-learn, Joblib

• Database: MySQL

• IDE: VS Code & Eclipse

• API Testing Tool: Frontend Integration (React UI)

5. Proposed System

The proposed Employee Attrition Prediction System provides a centralized platform for managing employee data and predicting attrition.

Employee Management: Manage employee records including personal and job-related details

Prediction System: Predict whether an employee will leave or stay using machine learning

Flask API Integration: Connect backend with ML model for real-time predictions

Data Centralization: Store all employee data in MySQL database

Risk Analysis: Provide probability score and risk levels (High, Medium, Low)

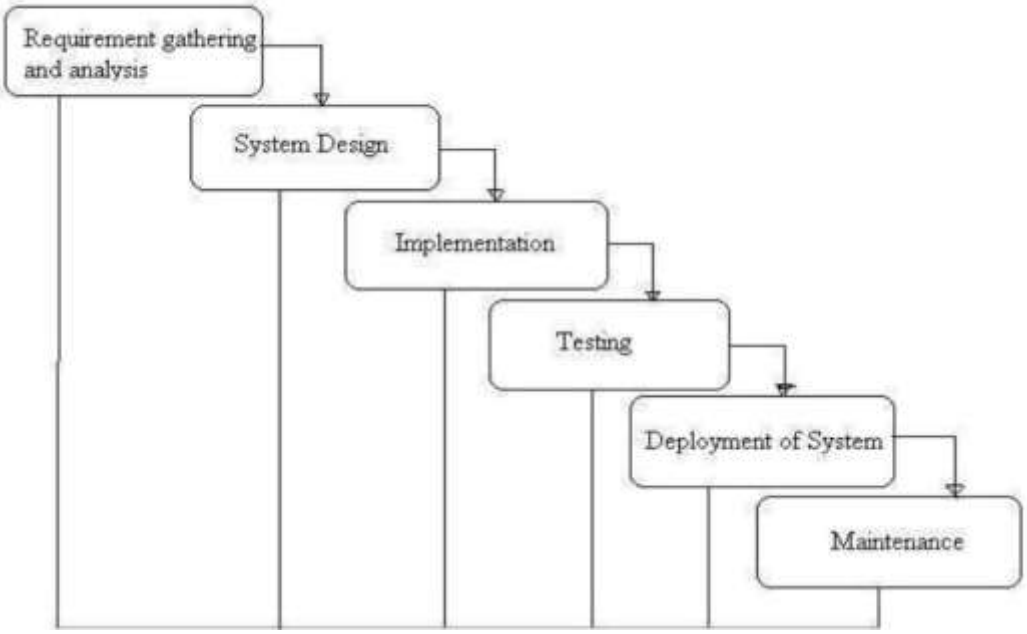


Fig. 5.1 . Phases of Development

6. Phases of Development

1. Requirement Analysis & System Study

Identify challenges in employee attrition

Gather requirements from HR perspective

Define system functionalities

2. System Design

Design database schema for employee data

Plan system architecture (React → Spring Boot → Flask → ML Model)

Define API structure

3. Implementation (Coding)

Develop frontend using React

Develop backend using Spring Boot

Build ML model using Python (Random Forest)

Create Flask API for prediction

4. Testing & Debugging

Unit testing of each module

Integration testing of full system

Debugging model predictions and API responses

5. Deployment & Maintenance

Deploy application locally or on server

Maintain model updates and system improvements

Fix bugs and enhance performance.

7. ER Diagram

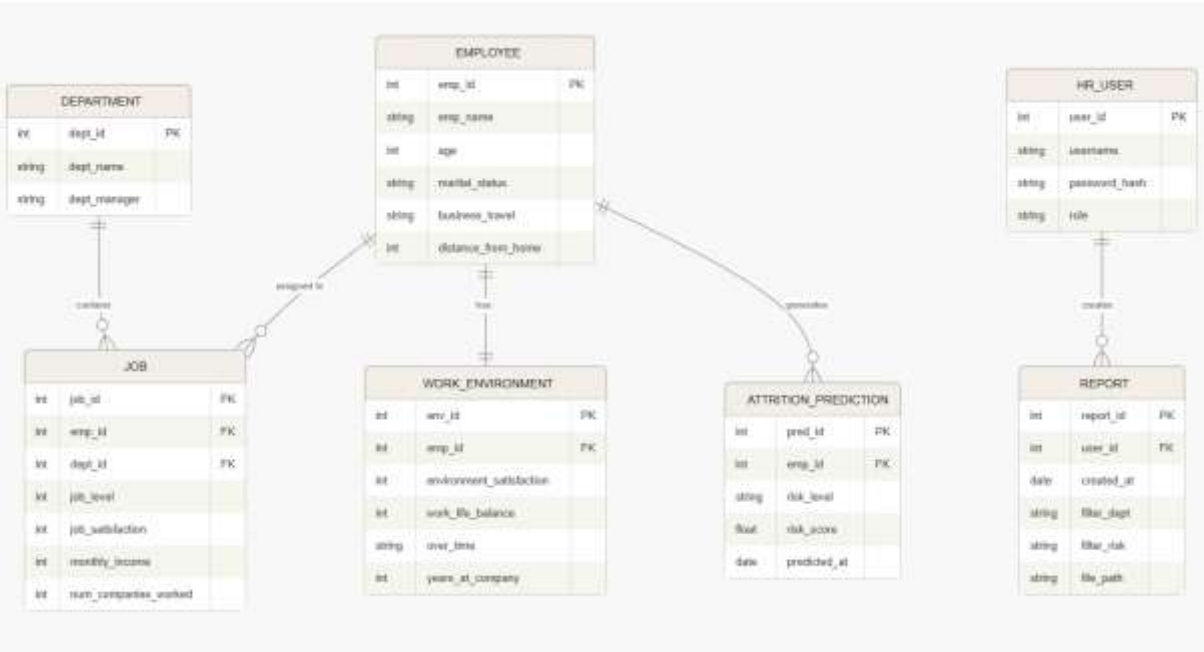


Fig. 7.1. ER Diagram

The ER diagram for the Employee Attrition Prediction System represents the logical structure of the database, showing entities, their attributes, and relationships among them. It provides a clear view of how employees, departments, job details, work environment factors, predictions, and HR users interact within the system.

Entities and Their Attributes:

1. Employee

Attributes:

- emp_id (PK)
- emp_name
- age
- marital_status
- business_travel
- distance_from_home

Description:
Represents the core entity whose attrition risk is being analyzed. It acts as the central entity connecting all other modules in the system.

2. Department

Attributes:

- dept_id (PK)
- dept_name
- dept_manager

Description:
Stores organizational department information used for grouping employees and generating department-level analytics.

3. Job

Attributes:

- job_id (PK)
- emp_id (FK)
- dept_id (FK)
- job_level
- job_satisfaction
- monthly_income
- num_companies_worked

Description:
Captures job-specific attributes of an employee that are critical input features for the machine learning attrition prediction model.

4. Work Environment

Attributes:

- env_id (PK)
- emp_id (FK)
- environment_satisfaction
- work_life_balance
- over_time
- years_at_company

Description:
Stores workplace condition factors that directly influence an employee's likelihood of leaving the organization.

5. Attrition Prediction

Attributes:

- pred_id (PK)
- emp_id (FK)
- risk_level
- risk_score
- predicted_at

Description:
Records each machine learning prediction result for employees over time, including risk classification (Low / Medium / High) and probability score.

6. HR User

Attributes:

- user_id (PK)
- username
- password_hash
- role

Description:
Represents HR system users (Admin/Manager) who access the platform, manage records, and generate reports.

7. Report

Attributes:

- report_id (PK)
- user_id (FK)
- created_at

- filter_dept
- filter_risk
- file_path

Description:
Tracks generated attrition risk reports, including applied filters, creation timestamp, and exported file location.

2. Relationships

- Employee → Job:
  One employee is assigned to one job record, while a department can contain multiple job records (1 : M).
- Department → Job:
  One department can have multiple job assignments (1 : M).
- Employee → Work Environment:
  Each employee has exactly one work environment record (1 : 1).
- Employee → Attrition Prediction:
  One employee can have multiple prediction records over time (1 : M).
- HR User → Report:
  One HR user can generate multiple reports with different filters (1 : M).

3. ER Diagram Summary

- The Employee entity is the central component of the system, connected to Job details, Work Environment, and Attrition Predictions.
- The Job table acts as a bridge between Employee and Department, helping normalize role-related attributes.

- The Attrition Prediction table maintains historical records of predictions, enabling trend analysis over time.
- The database structure follows Third Normal Form (3NF), ensuring minimal redundancy and high data integrity.
- This ER design supports:
    - Efficient HR analytics
    - Real-time machine learning predictions
    - Role-based access control
    - Report generation across all system modules

3. System Design

A) Modules

1. Employee Module

This module manages all employee-related information used for prediction.

Users can:

- Add, and delete employee records
- Store details like age, department, salary, job role, etc.
- View employee profiles
- Send employee data for attrition prediction

2. Prediction Module (ML Integration)

This is the core module responsible for predicting employee attrition.

Users can:

- Send employee data to the ML model via API
- Receive prediction results such as:
    - Attrition (Yes/No)
    - Probability score
    - Risk level (Low / Medium / High)
- View prediction history
-

3. Model Training Module

This module handles training and updating the machine learning model.

It includes:

- Loading dataset (HR Employee dataset)
- Data preprocessing:
  - Removing unnecessary columns
  - Encoding categorical values
  - Handling class imbalance (oversampling)
- Training model (Random Forest)

Users/Admin can:

- Retrain model with new data
- Improve prediction accuracy

4. API Integration Module (Flask Service)

This module connects the ML model with Spring Boot backend.

It includes:

- REST API built using Flask
- Loading model artifacts:
  - attrition_model.pkl
  - scaler.pkl
  - encoders
- Data preprocessing before prediction
- Returning JSON response to backend

Endpoints:

- POST /predict → Predict attrition
- GET /health → Check API status

5. Backend Module (Spring Boot)

This module manages business logic and communication between frontend and ML API.

Users can:

- Submit employee data from UI
- Call Flask API for prediction
- Store prediction results in database
- Manage employee and prediction records

Responsibilities:

- REST APIs
- Service layer logic
- Database interaction (JPA/Hibernate)
- Integration with Flask API

6. Frontend Module (React)

This module provides user interface for the system.

Users can:

- Enter employee details via forms
- View prediction results (Yes/No, probability, risk level)
- View dashboards and reports

Features:

- Form validation
- API integration with Spring Boot
- Dynamic UI updates

7. System Module (Backend Services)

This module acts as the backbone of the system.

It handles:

- CRUD operations (Employee, Prediction)
- Communication between:
  - React → Spring Boot → Flask
- Data preprocessing and validation
- Model integration
- Ensuring data consistency

B) Data Structure of All Modules

We use a database named: **employee_attrition_system**

1. Customized Tables

Employee Table (employee)

Stores employee details used for prediction.

Attributes:

- emp_id (PK)
- age
- department
- job_role
- monthly_income
- work_life_balance
- job_satisfaction
- years_at_company

- overtime
- marital_status
- distance_from_home

Prediction Table (prediction)

Stores prediction results for employees.

Attributes:

- pred_id (PK)
- emp_id (FK)
- prediction (Yes/No)
- probability
- risk_level (Low/Medium/High)
- prediction_date

Foreign Key:

- emp_id → references employee(emp_id)

Model Metadata Table (optional)

Stores model-related information.

Attributes:

- model_id
- model_name (Random Forest)
- accuracy
- roc_auc
- trained_on_date

2. System Architecture Flow

Flow:

1. User enters employee data in React
2. React sends request to Spring Boot
3. Spring Boot:
    - Validates data
    - Calls Flask API (/predict)
4. Flask API:
    - Preprocesses data
    - Uses trained model
    - Returns prediction
5. Spring Boot:
    - Stores result in database
    - Sends response to frontend
6. React displays:
    - Prediction
    - Probability
    - Risk Level

Screenshots:

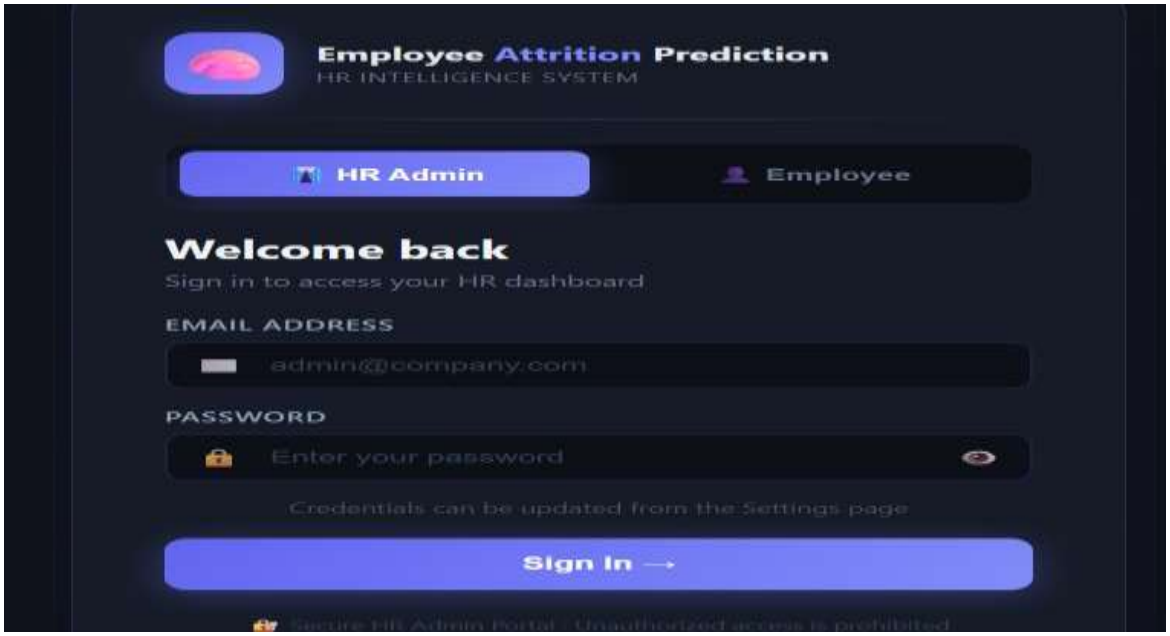


Fig.3.1 User & Admin Authentication

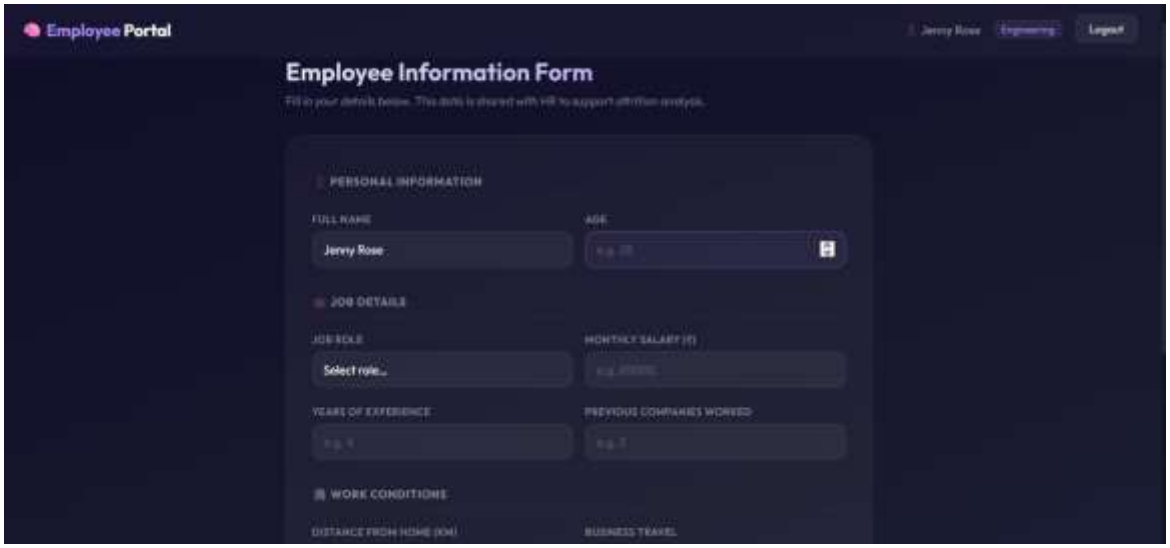


Fig.3.2 Employee Information Form

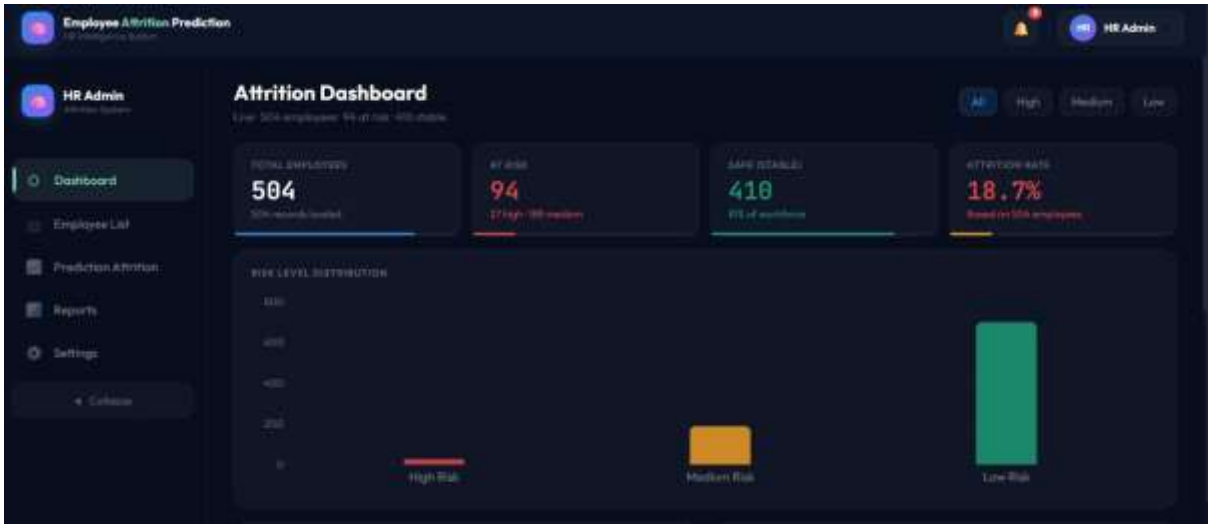


Fig.3.3 HR Analytics Dashboard

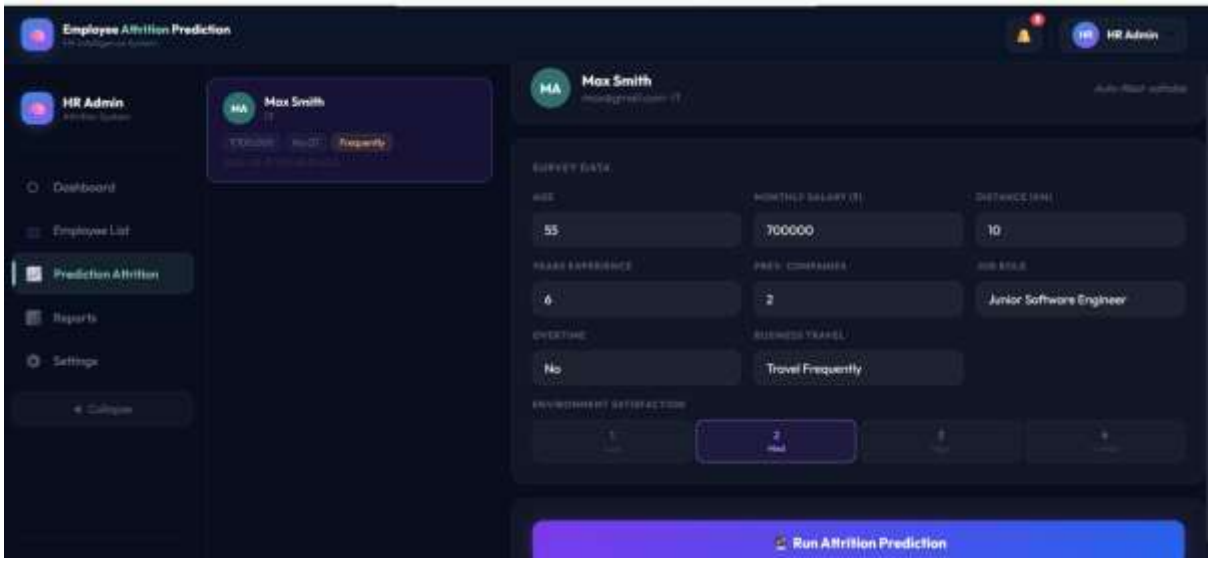


Fig. 3.4. Add Employee

Fig. 3.5. Attrition Prediction Result

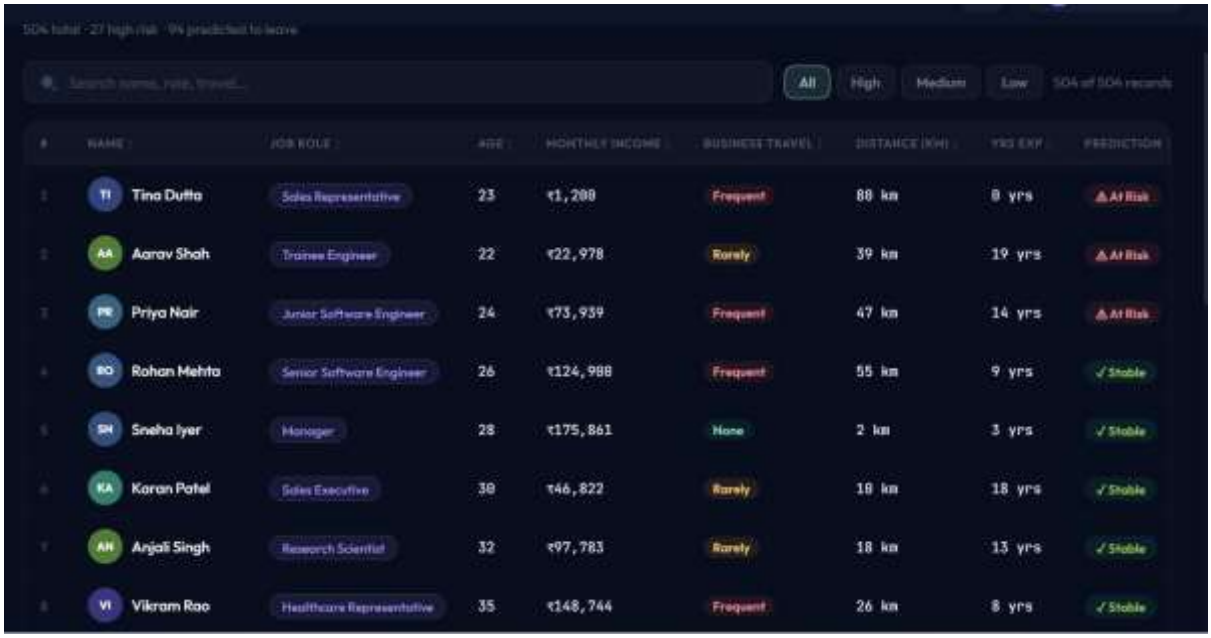
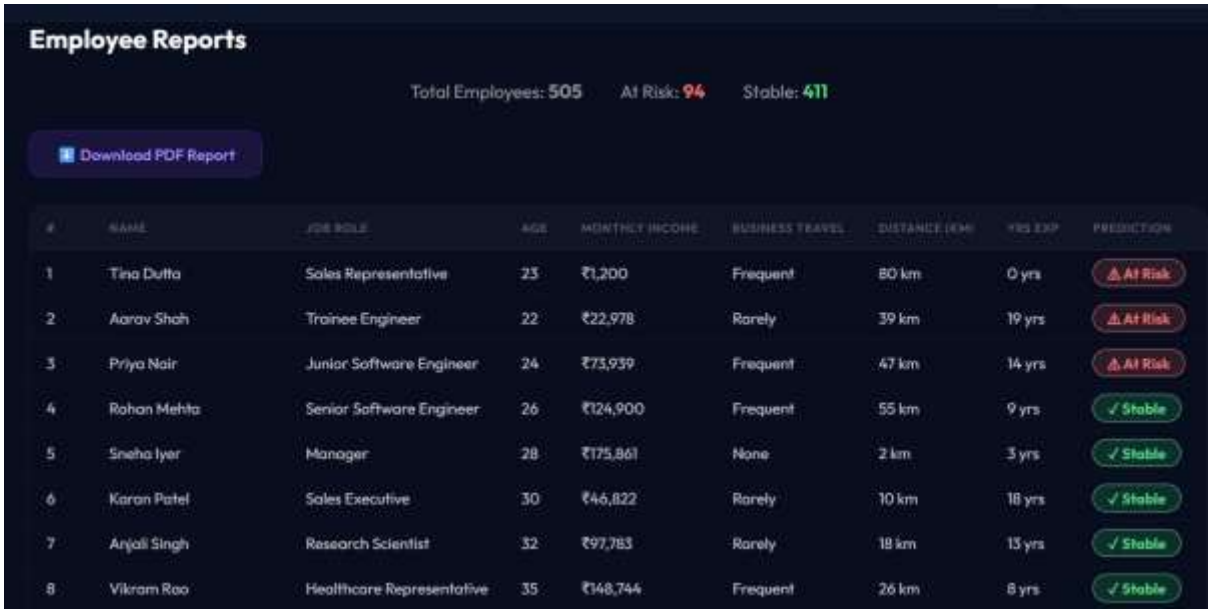


Fig.3.6.Employee List



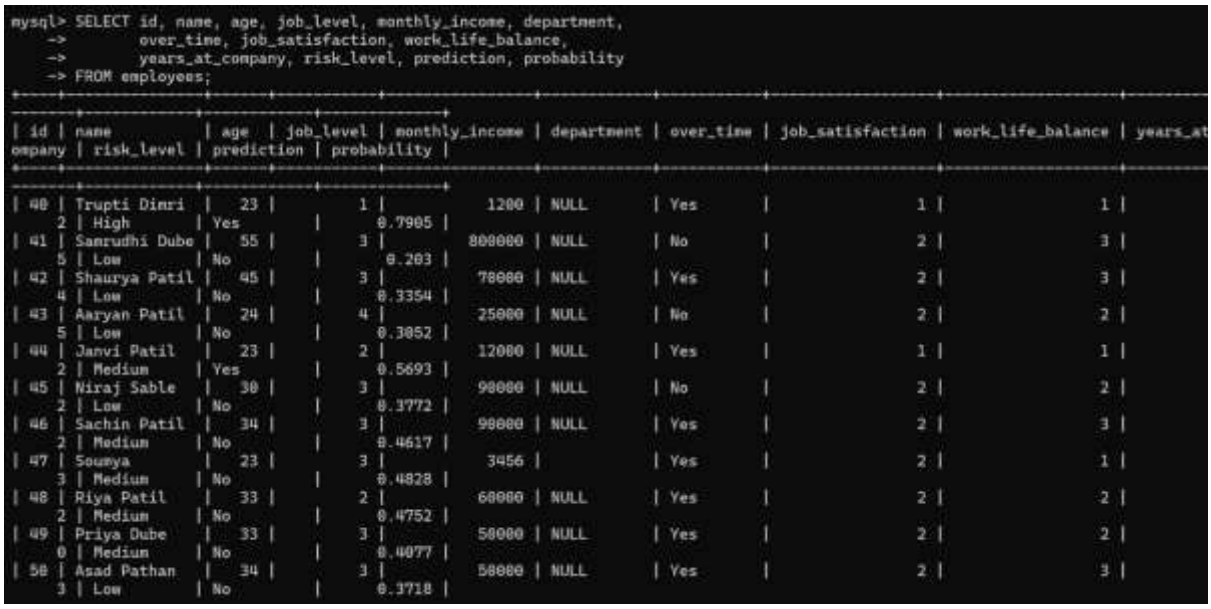
Employee Reports

Total Employees: 505 At Risk: 94 Stable: 411

Download PDF Report

#	NAME	JOB ROLE	AGE	MONTHLY INCOME	BUSINESS TRAVEL	DISTANCE (KM)	YRS EXP	PREDICTION
1	Tina Dutta	Sales Representative	23	₹1,200	Frequent	80 km	0 yrs	⚠ At Risk
2	Aarav Shah	Trainee Engineer	22	₹22,978	Rarely	39 km	19 yrs	⚠ At Risk
3	Priya Nair	Junior Software Engineer	24	₹73,939	Frequent	47 km	14 yrs	⚠ At Risk
4	Rohan Mehta	Senior Software Engineer	26	₹124,900	Frequent	55 km	9 yrs	✓ Stable
5	Sneha Iyer	Manager	28	₹175,861	None	2 km	3 yrs	✓ Stable
6	Karan Patel	Sales Executive	30	₹46,822	Rarely	10 km	18 yrs	✓ Stable
7	Anjali Singh	Research Scientist	32	₹97,783	Rarely	18 km	13 yrs	✓ Stable
8	Vikram Rao	Healthcare Representative	35	₹148,744	Frequent	26 km	8 yrs	✓ Stable

Fig.3.7.Report Generation



```
mysql> SELECT id, name, age, job_level, monthly_income, department,
    ->        over_time, job_satisfaction, work_life_balance,
    ->        years_at_company, risk_level, prediction, probability
    -> FROM employees;
+----+--------------+-------+-----------+----------------+-------------+-----------+------------------+------------------+---------
-------+-------------+------------+--------------+
| id | name         | age   | job_level | monthly_income | department | over_time | job_satisfaction | work_life_balance | years_at
ompany | risk_level | prediction | probability |
+----+--------------+-------+-----------+----------------+-------------+-----------+------------------+------------------+---------
-------+-------------+------------+--------------+
| 40 | Trupti Dimri  |   23 |         1 |           1200 | NULL        | Yes       |                1 |                1 |
     2 | High        | Yes        |       0.7905 |
| 41 | Samrudhi Dube |   55 |         3 |         800000 | NULL        | No        |                2 |                3 |
     5 | Low         | No         |        0.203 |
| 42 | Shaurya Patil |   45 |         3 |          78000 | NULL        | Yes       |                2 |                3 |
     4 | Low         | No         |       0.3354 |
| 43 | Aaryan Patil  |   24 |         4 |          25000 | NULL        | No        |                2 |                2 |
     5 | Low         | No         |       0.3052 |
| 44 | Janvi Patil   |   23 |         2 |          12000 | NULL        | Yes       |                1 |                1 |
     2 | Medium      | Yes        |       0.5693 |
| 45 | Niraj Sable   |   30 |         3 |          90000 | NULL        | No        |                2 |                2 |
     2 | Low         | No         |       0.3772 |
| 46 | Sachin Patil  |   34 |         3 |          90000 | NULL        | Yes       |                2 |                3 |
     2 | Medium      | No         |       0.4617 |
| 47 | Soumya        |   23 |         3 |           3456 |             | Yes       |                2 |                1 |
     3 | Medium      | No         |       0.4828 |
| 48 | Riya Patil    |   33 |         2 |          60000 | NULL        | Yes       |                2 |                2 |
     2 | Medium      | No         |       0.4752 |
| 49 | Priya Dube    |   33 |         3 |          50000 | NULL        | Yes       |                2 |                2 |
     0 | Medium      | No         |       0.4077 |
| 50 | Asad Pathan   |   34 |         3 |          50000 | NULL        | Yes       |                2 |                3 |
     3 | Low         | No         |       0.3718 |
```

Fig.3.8.MySQL Database Schema

```
▲ Training Random Forest...

==================================================
MODEL EVALUATION
==================================================
   Accuracy  : 0.8401
   ROC-AUC   : 0.7840   ← key metric for imbalanced data

Confusion Matrix:
   True No   (correct stay)   : 233
   False Yes (wrongly flagged): 14
   False No  (missed leavers): 33   ← want this LOW
   True Yes  (caught leavers): 14   ← want this HIGH

Classification Report:
              precision    recall  f1-score   support

          No       0.88      0.94      0.91       247
         Yes       0.50      0.30      0.37        47

    accuracy                           0.84       294
   macro avg       0.69      0.62      0.64       294
weighted avg       0.82      0.84      0.82       294

🔑 TOP 15 FEATURES DRIVING ATTRITION:
   1. MonthlyIncome                        0.0782
   2. Age                                  0.0687
   3. TotalWorkingYears                    0.0599
   4. DailyRate                            0.0562
   5. OverTime                             0.0557
   6. YearsAtCompany                       0.0472
```

```
📝 PROFILE TESTS:
  ✅ CORRECT | HIGH risk  → expect Yes: Yes (82.2%)
  ⚠️ CHECK | MEDIUM risk → expect Yes: Yes (59.1%)
  ✅ CORRECT | LOW risk   → expect No : No (28.1%)

✅ All files saved:
   attrition_model.pkl | scaler.pkl | train_columns.pkl
   categorical_encoders.pkl | label_encoder.pkl

D:\AttritionProject>python predict.py
✅ Model loaded | Expecting 30 features:
   ['Age', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHome', 'Education', 'EducationField', 'EnvironmentSatisfaction',
 'Gender', 'HourlyRate', 'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction', 'MaritalStatus', 'MonthlyIncome', 'MonthlyRate',
 'NumCompaniesWorked', 'OverTime', 'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction', 'StockOptionLevel', 'TotalWo
rkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion', 'YearsWit
hCurrManager']

==================================================
SINGLE EMPLOYEE PREDICTION
==================================================
  Prediction  : Yes
  Probability : 50.2%
  Risk Level  : Medium 🟡
==================================================
```

Fig. 3.9. ML Model Evaluation & Deployment

4. Implementation

The implementation phase involves converting the system design into a fully functional software application. The Employee Attrition Prediction System is developed as a web-based application using React for the frontend, Spring Boot for backend services, Python for machine learning, and MySQL as the relational database. This section describes the technologies, logic, and design used in the implementation.

1. Technology Stack

• Frontend: React (for building interactive user interface)

• Backend: Spring Boot (Java) for REST API and business logic

• Machine Learning: Python

• ML Framework: Scikit-learn

• API Framework: Flask (for exposing ML model as REST API)

• Database: MySQL (Relational Database Management System)

• Libraries: Pandas, NumPy, Joblib

• Development Environment: VS Code And Eclipse

• API Testing: Frontend Integration (React UI)

2. Backend Implementation (Spring Boot)

The backend handles business logic and communication between frontend and machine learning model.

Key Functionalities:

• Employee Management:

Add, update, view, and delete employee records

• API Integration:

Spring Boot sends employee data to Flask API and receives prediction results

• Data Processing:

Handles request/response conversion in JSON format

• Database Operations:

Stores employee data and prediction results in MySQL

3. Machine Learning Implementation (Python + Flask)

The machine learning module is responsible for training the model and predicting employee attrition.

Model Development:

• Dataset is loaded and preprocessed using Pandas and NumPy

• Unnecessary columns are removed

• Categorical data is encoded using Label Encoding

• Feature scaling is applied using StandardScaler

• Imbalanced dataset is handled using oversampling technique

Model Used:

• Random Forest Classifier

Handles non-linear relationships

Provides high accuracy

Works well with structured HR data

Model Evaluation:

• Accuracy Score

• ROC-AUC Score

• Confusion Matrix

• Classification Report

Model Deployment:

• Model is saved using Joblib (.pkl files)

• Flask API is used to expose prediction endpoint (/predict)

• API returns:

Prediction (Yes/No)

Probability Score

Risk Level (High / Medium / Low)

4. Frontend Implementation (React)

The frontend provides a user-friendly interface for HR users.

Features:

• Form to input employee details

• Sends data to backend via API calls

• Displays prediction results dynamically

• Provides clean and responsive UI

5. Database Implementation (MySQL)

The database is implemented using MySQL with the following main tables:

• Employee – Stores employee details

• Department – Stores department information

• Salary – Stores salary and bonus details

• Project – Stores project information

• Employee_Project – Junction table for employee-project relationship

• Prediction – Stores attrition prediction results

Relationships:

• One Department → Many Employees

• One Employee → One Salary

• One Employee ↔ Many Projects (M : N)

• One Employee → One Prediction

6. API Integration (Spring Boot + Flask)

• React sends request → Spring Boot

• Spring Boot sends data → Flask API

• Flask API processes data using ML model

• Prediction result returned → Spring Boot → React

This ensures real-time prediction functionality.

7. Security Implementation

• Data Validation: Input validation at frontend and backend

• Exception Handling: Handles invalid inputs and API failures

• Transaction Management: Ensures data consistency in database

• Foreign Key Constraints: Maintain relational integrity

• CORS Handling: Flask-CORS used for secure API communication

5. Results and Discussion

1. Results

o Functional Results

- **Employee Module:**
  Users can add, view, update, and delete employee records. Employee data is stored accurately and is effectively used for prediction.
- **Department Module:**
  Departments are managed efficiently, and employees are correctly linked using foreign key relationships.
- **Salary Module:**
  Salary details, including monthly salary and bonuses, are stored and properly associated with employees.
- **Project Module:**
  Employees can be assigned to multiple projects, and their roles are managed through the **Employee_Project** relationship.
- **Prediction Module:**
  The system successfully predicts whether an employee is likely to leave or stay using a machine learning model. It provides:
    - Prediction (Yes/No)
    - Probability Score
    - Risk Level (High / Medium / Low)
- **API Integration:**
  Smooth communication between **React, Spring Boot, and Flask API** ensures real-time prediction results.
- **Database Operations:**
  All CRUD operations execute successfully using **MySQL**, ensuring consistent and reliable data storage.

o Non-Functional Results

- **Usability:**
  The web-based interface is user-friendly and easy to navigate, allowing HR users to interact with the system efficiently.
- **Performance:**
  Prediction results are generated quickly, and API communication between system components is efficient.
- **Reliability:**
  Proper exception handling ensures smooth system performance even when invalid inputs are provided.
- **Data Integrity:**
  Relationships between entities (Employee, Department, Salary, Project, Prediction) are maintained using foreign key constraints.
- **Scalability:**
  The modular architecture (**React + Spring Boot + Flask + ML**) allows easy addition of new features such as dashboards and analytics.

2. Discussion

1. Achievement of Objectives

- The system successfully predicts employee attrition using machine learning.
- All major functionalities, such as employee management and prediction, are implemented effectively.
- Integration between frontend, backend, and machine learning model is achieved successfully.

2. User Experience

- The React-based interface provides a smooth and interactive user experience.
- Real-time prediction results enhance usability.
- Simple forms and clear output make the system easy to use.

3. Challenges Faced

- Handling imbalanced datasets required the implementation of oversampling techniques.
- Integrating **Spring Boot with Flask API** required proper request handling and data formatting.
- Encoding categorical data and maintaining consistency between training and prediction phases was challenging.
- Ensuring accurate feature alignment between model training and real-time prediction required careful implementation.

4. Limitations

- The system depends on the quality of the dataset for accurate predictions.
- Limited visualization features (no advanced dashboards or graphical reports).
- No authentication module for secure user access.
- Model accuracy may vary with different datasets.

5. Future Enhancements

- Add advanced data visualization dashboards (charts and graphs).
- Implement user authentication and role-based access control.

- Improve model accuracy using advanced algorithms (e.g., XGBoost, Deep Learning).
- Deploy the system on cloud platforms for better scalability.
- Integrate automated alerts for identifying high-risk employees.

6.Conclusion

The development of the **Employee Attrition Prediction System** has successfully demonstrated the design and implementation of an intelligent, efficient, and scalable platform for predicting employee turnover.

The system enables users to:

- Manage employee data effectively through a centralized system
- Analyze employee-related factors such as salary, experience, and job satisfaction
- Predict employee attrition using a machine learning model
- Obtain real-time prediction results with probability scores and risk levels

Key Achievements

- Seamless integration between frontend (React), backend (Spring Boot), and machine learning module (Python Flask), ensuring smooth data flow and real-time predictions
- Implementation of a Random Forest machine learning model along with data preprocessing techniques such as encoding, scaling, and handling imbalanced data to improve accuracy
- Efficient database design using MySQL, maintaining structured relationships among entities such as Employee, Department, Salary, Project, and Prediction
- Development of a modular architecture that enhances scalability and allows easy integration of additional features such as dashboards and analytics

- Robust API communication and proper exception handling, ensuring system reliability and consistent performance

Overall, the Employee Attrition Prediction System provides a smart and data-driven solution for identifying employees at risk of leaving an organization. It reduces manual effort, improves decision-making, and enables organizations to take proactive steps to enhance employee retention.

Furthermore, the system establishes a strong foundation for future expansion into advanced analytics platforms, cloud deployment, and enterprise-level workforce management solutions.

7. Future Scope

Although the Employee Attrition Prediction System effectively achieves its current objectives, several enhancements can be implemented to make the system more advanced and user-friendly in the future:

1. Advanced Web Interface

Develop a more advanced and responsive web interface with improved UI/UX design.

Integrate dashboards with charts and graphs for better visualization of employee data and prediction results.

2. Authentication & Role Management

Implement secure login and authentication features for different users such as HR, Admin, and Managers.

Introduce role-based access control to restrict access to sensitive employee data and prediction results.

3. Advanced Machine Learning Models

Improve prediction accuracy by using advanced algorithms such as XGBoost, Gradient Boosting, or Deep Learning models.

Implement continuous model training using real-time data to enhance performance over time.

6. Cloud Deployment

Deploy the application on cloud platforms (AWS, Azure, or GCP) for better scalability and accessibility.

Enable multi-user access and real-time data synchronization.

8. Data Security Enhancements

Implement encryption techniques to secure sensitive employee data.

Add audit logs and monitoring to track system usage and ensure data privacy.

8. Bibliography and References

In preparing this project report on the Employee Attrition Prediction System, various books, research papers, and online resources were referred to. These sources provided valuable insights into machine learning, system design, database management, and full-stack development.

Books

- Ethem Alpaydin – *Introduction to Machine Learning* https://mitpress.mit.edu/9780262043793/introduction-to-machine-learning/
- Aurélien Géron – *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/
- Abraham Silberschatz, Henry Korth, S. Sudarshan – *Database System Concepts* https://www.mheducation.com/highered/product/database-system-concepts-silberschatz/M9780078022159.html
- Ian Sommerville – *Software Engineering* https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003437Research Papers / Journals

Research Papers / Journals

- International Journal of Computer Applications https://www.ijcaonline.org/
- IEEE Xplore Digital Library https://ieeexplore.ieee.org/